



THE UNIVERSITY OF BRITISH COLUMBIA - OKANAGAN CAMPUS

COSC 488 – Scott Fazackerley
Directed Studies Report

Sam Sikora (55017370) combination with Alex Taschuk

Introduction

The Transmission Control Protocol (TCP) is a connection-oriented Internet protocol that offers reliable, ordered, error-checked, and congestion-aware delivery of data. TCP offers applications full-duplex transmission of data in the form of a stream of bytes. Optimizing the parameters and algorithms involved in TCP has been a hot research topic for many years, as any improvements to the service could offer ample benefits to the entire internet. One such area of research is congestion control.

Congestion control algorithms (CCAs) seek to utilize the network's bandwidth as much as possible without overloading it to the point of failure. To achieve these goals in parallel, congestion control algorithms send data in a "saw-tooth" pattern, where the amount of data increases as it probes the network for more bandwidth, and on a failure event (either triple ACK or timeout), drops the flow of data back down to probe again. Many CCAs have been introduced to try to balance both goals. These classical CCAs are fundamentally reactive; they rely on packet loss as a primary signal of network saturation. However, there is little research into a model that could actively predict failure events and adjust accordingly before these events occurred.

The development of a predictive CCA would require analysis of a large amount of data to uncover unknown, complex, data relationships. Machine learning offers a natural fit for this problem. The promise is that given enough training data, a model could learn the subtle warning signs to congestion and alert before they happen. In particular, Long Short-Term Memory (LSTM) architectures are well-suited to this domain. LSTMs are a type of recurrent neural network designed specifically to learn both long running and short-term data relationships across time, making them a perfect fit to analyze patterns that unfold over many timestamps, and a ideal candidate to develop a predictive CCA.

First, we introduce UTCP, a custom user space TCP implementation built to provide the extensibility and structured logging necessary to generate the training data a machine learning model requires. We benchmark UTCP against the classical congestion control algorithms, Tahoe, Reno, and NewReno to validate the performance of the implementation. We then present the design and training of an LSTM-based congestion predictor and evaluate the model. Finally, we discuss the central limitation uncovered by live deployment, inference latency, and outline the

steps required to close the gap between a promising research result and a deployable congestion control component.

UTCP

Introduction

The User Datagram Protocol (UDP) is a transport layer protocol that offers no guarantees about delivery. Data is wrapped in distinct packets and sent to the receiver over an Internet Protocol (IP) network. Therefore, UDP has the same distinct challenges of the underlying IP network: packet loss, out-of-order delivery, and receiver capacity to accept a message. The only additional property provided by UDP is a checksum and the capability of a packet to arrive at the receiver through a specific port. These properties make UDP a justified proxy for IP. Mirroring how the kernel places TCP segments into IP packets, UTCP achieves user-space TCP by embedding TCP segments inside UDP packets. Notably, this means that for every UTCP packet sent, there are eight bytes of overhead in the UDP header.

A subtlety of this approach is that the port numbers inside the TCP header lose their conventional meaning. Because the outer UDP packet has already routed the data to the correct port on the receiver, the inner TCP ports are no longer needed for OS-level delivery. One way to handle this extra data is to ignore the TCP port numbers, though this would restrict a UTCP process to a single connection at a time. A better approach is to repurpose the port system within UTCP itself. When a segment arrives at the underlying UDP port, UTCP can route it to virtual, user-space ports. The new port number meaning enables multiple applications to send and receive data concurrently through a single UDP port and UTCP process. Since we handle many different connections, each connection will be assigned a Transmission Control Buffer (TCB) that holds state information such as the next segment to send, congestion window, TCP timers, etc.

API

UTCP is designed as a drop-in replacement for standard TCP, so the package exposes functions that closely mimic the traditional Berkeley Sockets API. This allows application developers to integrate UTCP easily:

- `utcp_socket()` - Creates a new UTCP socket and returns a virtual file descriptor referencing it.
- `utcp_bind(fd, address)` - Binds a socket address to a UTCP socket. This populates the local properties of the socket, such as the source IP and the virtual port it should listen on.
- `utcp_connect(fd, address)` - Connects to a remote UTCP application at the specified address. This function blocks until the TCP three-way handshake is successfully established.
- `utcp_listen(fd)` - Marks the UTCP socket as a passive socket, indicating it is ready to accept incoming connection requests.
- `utcp_accept(fd)` - Blocks and waits for an incoming UTCP connection to be established on the listening socket.
- `utcp_send(fd, buffer, length)` - Writes data from the application buffer into the UTCP send buffer to be transmitted to the connected receiver.
- `utcp_read(fd, buffer, length)` - Reads incoming data from the UTCP receive buffer into the application's buffer.

System Architecture

The system utilizes a multi-threaded architecture to process network events, application requests, and track timers concurrently. To achieve the asynchronous TCP behavior, responsibilities are divided across the following threads:

- **Main Thread:** This thread executes the user's main program and interacts with UTCP through the API functions mentioned in Section 2.b. This thread manages sockets and will copy and write data from the UTCP package to the application space.

- **Listen Thread:** Once initialized on UTCP start up, this thread is a dedicated background thread to listen for incoming UDP packets on the single host port. Upon packet arrival, the thread will verify it is a UTCP packet, then it will use the source/destination 4-tuple to find the corresponding TCB and process the segment accordingly.
- **Timer Thread:** The timer thread wakes up at fixed intervals to iterate through all the active TCBs and decrement their timers. If a timer expires, this thread will call the required expiration logic.

Instead of a dedicated sender thread, any of the three threads can invoke the `utcp_output` function on state changes (Main Thread queues new data, Listen Thread generates an ACK, Timer Thread triggers a timeout retransmission) to invoke a segment to be sent. The engine evaluates the current state of the TCB to determine if a segment can and how it should be sent. A call to `utcp_output` may generate zero, one, or many segments. On RFC required transmissions, such as receiving an out of order segment, the `TF_ACKNOW` flag is set to guarantee at least one acknowledgement is sent to the receiver.

Feature List

The following features of TCP have been implemented in the UTCP program:

- Three-Way Handshake
- TCP Finite State Machine
- Cumulative and Delayed Acknowledgments
- Out-of-Order (OOO) Reassembly
- Retransmission Timer (RTO)
- Flow Control
- Congestion Control (CC) algorithms including, TCP Tahoe, Reno, and New Reno.
- Slow Start and Congestion Avoidance
- Fast Retransmit and Fast Recovery
- Window Scaling

The following are features that are out of scope for current research but would be needed for feature-complete TCP.

- Connection Termination
- Keepalive & Persistence
- Checksum Verification (Omitted as the outer UDP packet contains a checksum)

Modular Congestion Control

The UTCP framework implements a fully decoupled, modular congestion control (CC) architecture. Within the TCB, the data structure maintaining the state of a TCP connection, there is a pointer to a single, swappable CC function corresponding to the active algorithm (e.g., Reno, Tahoe). When a congestion event occurs, such as a normal ACK, duplicate ACK, or timeout, this function is invoked with the specific event type and its associated arguments. Then it is the CC's responsibility to process that event and update the congestion state variables, such as the congestion window and the slow-start threshold, accordingly. The `utcp_output` engine will then respect those variables and ensure that data in flight never exceeds the congestion window specified and calculated by the algorithm.

Figures and Results

To test the UTCP implementation, a “Standard B2ats v2” virtual machine (VM) was hosted in Microsoft Azure in the Central Canada region and communicated with a 2020, 8GB Ram, M1 MacBook Air. A 1GB file was transferred between the MacBook Air and the VM. The following figures plot the congestion control window and congestion events (triple acknowledgement, timeout). The slow start threshold is set arbitrarily high to start, so the figures plot the window after the first congestion event.

An important note about the network is that the network link between the computers would often drop segments in groups. A theory on the cause of the burst loss is that the MacBook Air was connected to a home router through Wi-Fi. Wireless mediums are notoriously volatile, as many environmental factors can cause transient signal interference lasting several

milliseconds. This behavior especially hurt algorithms Tahoe and Reno, because they are unequipped to handle multiple packets drops. This vulnerability was a primary driving factor for the development of NewReno, which modifies the fast recovery phase to successfully handle partial acknowledgments and recover from multiple simultaneous drops.

Tahoe

TCP Tahoe is the foundational congestion control algorithm. Upon detecting a congestion event, such as a triple duplicate acknowledgment (ACK), the algorithm sets the slow start threshold (ssthresh) to half of the unacknowledged data in flight and drastically reduces the congestion window (cwnd) to a single segment, forcing a return to the Slow Start phase. However, Tahoe is ill-suited for this network because of the group drop problem mentioned above.

Often, a triple duplicate ACK is detected, and the first missing segment is fast-retransmitted. Yet, because multiple packets were dropped, the receiver cannot send back the large, expected cumulative ACK. Instead, it sends a partial ACK that only acknowledges data up to the next missing packet. At this point, Tahoe becomes stuck. Because the sender's cwnd was reduced to 1 and the network pipeline is drained, it cannot send new data, nor can it receive enough new duplicate ACKs to trigger another fast retransmit. This recurring failure sequence is illustrated in Figure 1 (note that due to the near-simultaneous nature of these events, the "3x Dup ACK" indicators obscure the 'Timeout' markers). The graph appears to lack a distinct Slow Start phase because the rapid succession of triple duplicate ACKs and subsequent timeouts continuously drives the ssthresh to a minimal value

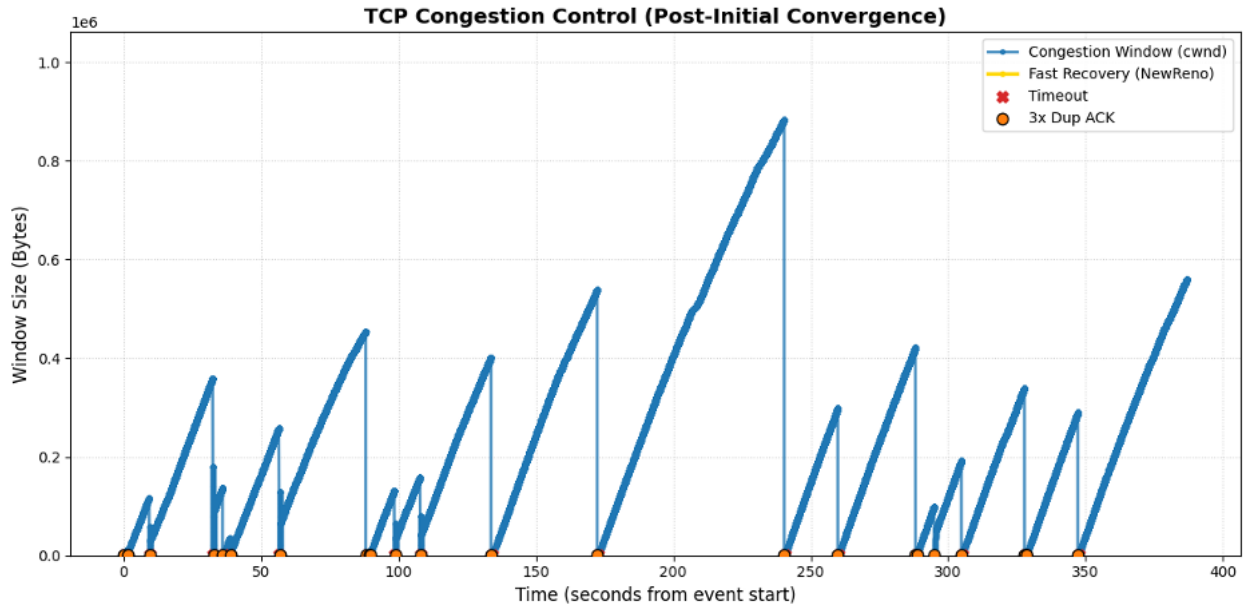


Figure 1: Congestion window size over time from a MacBook Air transmitting a 1GB file over a 6:50 minute period over a Wi-Fi connection using Tahoe.

Reno

Unlike Tahoe, Reno creates the assumption that a triple ACK event signals minor congestion in the network while a timeout signals major congestion. This algorithm differs from Tahoe upon triple ACK by setting the ssthresh and the cwnd to half of the data in flight. Testing demonstrates that while this Fast Recovery mechanism successfully maintains a higher volume of in-flight data, Reno remains vulnerable to burst packet losses. When multiple segments are dropped simultaneously, the protocol exhibits a similar failure sequence. Once the first triple ACK arrives, the packet is retransmitted. Subsequent ACKs then inflate the cwnd which sends new segments to the receiver. The receiver fills the first missing segment, but the following new data causes another set of triple ACKs for the next segment to be sent out. This triple ACK loop continues until the entire hole has been filled or a timeout occurs. The exception to this case is a transmission over Ethernet rather than Wi-Fi as shown in Figure 3. This supports the theory of burst loss resulting from a Wi-Fi connection, but more testing would need to be done to confirm this theory.

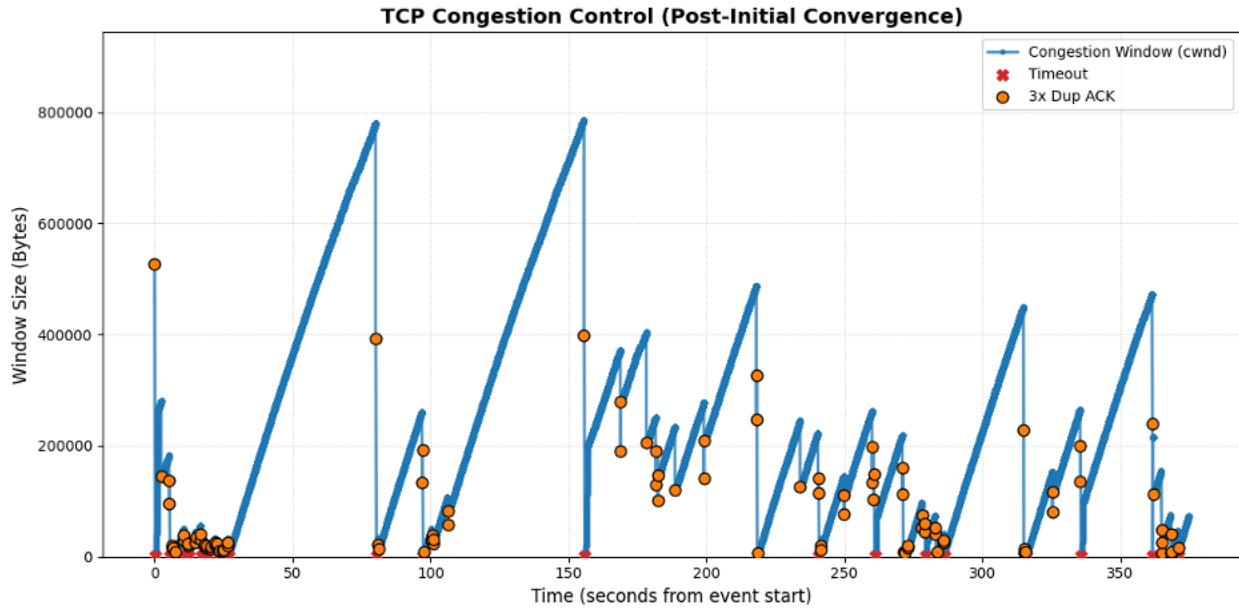


Figure 2: Congestion window size over time from a MacBook Air transmitting a 1GB file over a 6:16 minute period over a Wi-Fi connection using Reno.

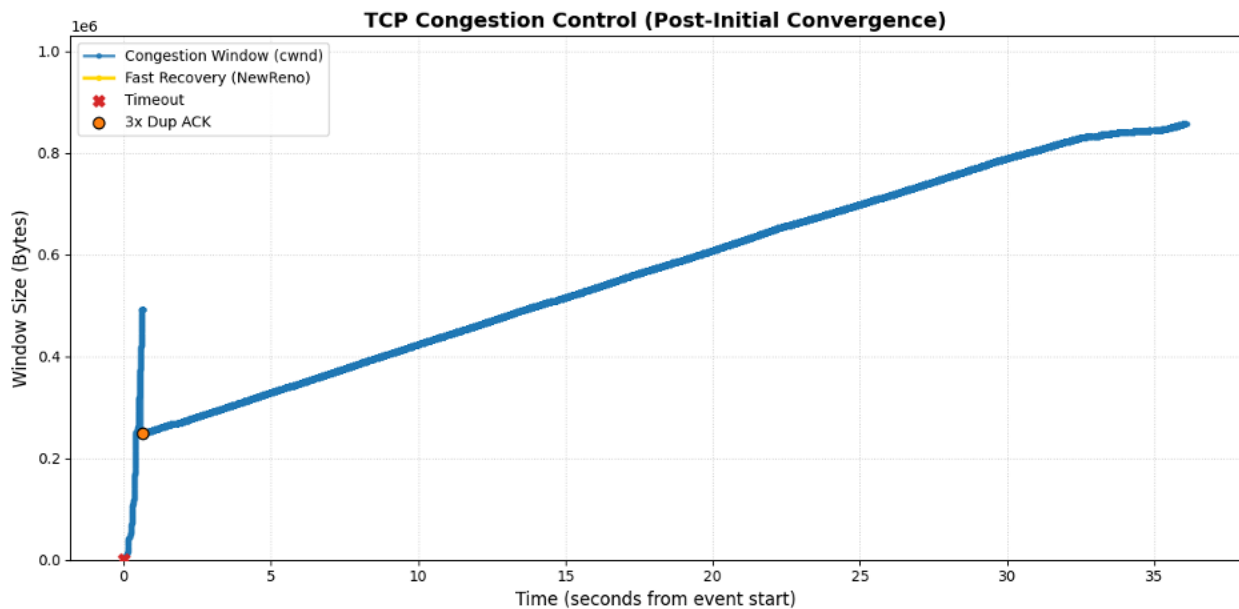


Figure 3: Congestion window size over time from a MacBook Air transmitting a 1GB file over a 1:48 minute period over an Ethernet connection using Reno.

NewReno

NewReno (standardized in RFC 6582) is an extension of Reno designed to address performance degradation during multiple packet losses. Unlike classic Reno, which exits Fast

Recovery upon receiving a partial acknowledgment, NewReno assumes the next segment was also lost, and immediately retransmits the next segment.

Upon detecting a congestion event via three duplicate ACKs, NewReno records the highest sequence number transmitted. This sequence number acts as a recovery threshold. The algorithm then enters Fast Recovery and retransmits the first missing segment. For every ACK that does not meet or exceed the threshold, NewReno assumes the partial ACK indicates that the next unacknowledged packet was also lost. It immediately retransmits the next missing segment. During this extended Fast Recovery phase, NewReno manages its congestion window (cwnd) to ensure data keeps flowing. It artificially inflates the cwnd for each duplicate ACK received, allowing new, out-of-order segments to be sent if the window permits. When a full acknowledgment arrives, it deflates the cwnd back to the slow-start threshold (ssthresh) and transitions into the Congestion Avoidance phase.

In our network, this algorithm fits perfectly. As shown by Figures 4 and 5, NewReno builds and drops the congestion window as expected, avoiding the catastrophic collapse seen in Tahoe or the repeating triple-ACK loops seen in Reno.

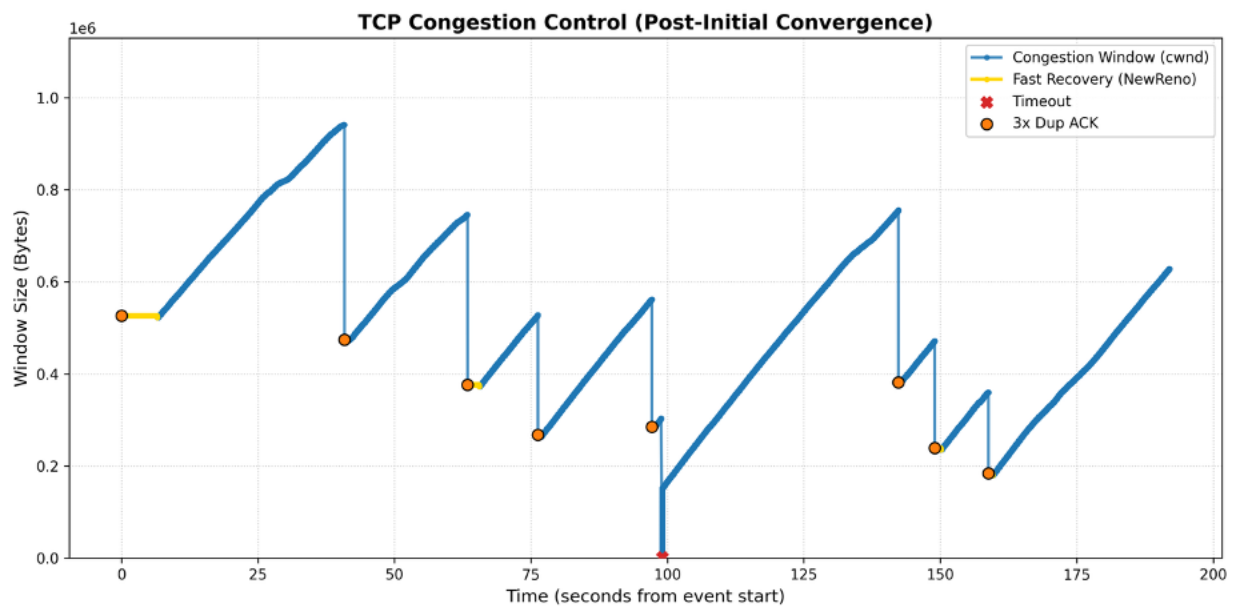


Figure 4: Congestion window size over time from a MacBook Air transmitting a 1GB file over a 3:19 minute period over a Wi-Fi connection using NewReno.

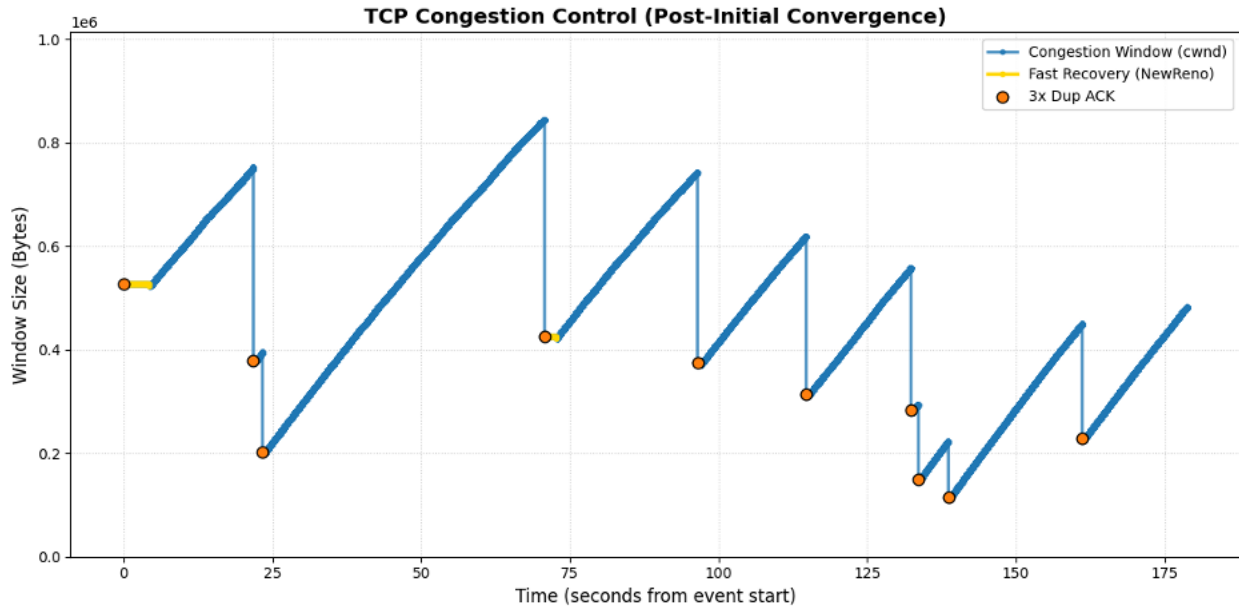


Figure 5: Congestion window size over time from a MacBook Air transmitting a 1GB file over a 3:14 minute period over a Wi-Fi connection using NewReno.

Predictive CCA

With a highly extendable codebase and structured logging now in place, we can initiate the next phase of the project: developing a predictive Congestion Control Algorithm (CCA). Our primary objective is to utilize a Long Short-Term Memory (LSTM) network to detect the early onset of network congestion. This model allows the CCA to dynamically adjust its pacing or window size to mitigate congestion before packet loss occurs.

The first design challenge is accurately defining an anomaly. A natural first instinct is to flag anomalous round-trip times (RTTs). If RTTs spike, congestion may be building, and a loss event could be imminent. The issue with that reasoning is classical CCAs already account for RTT jitter through Smoothed RTT and RTT Variation. Therefore, flagging a high RTT as anomalous would risk throttling a healthy connection that could have absorbed the jitter on its own. Rather, the only flag we have to alert us that the network was congested to a fatal amount is the congestion events: triple duplicate ACK and a timeout. These events directly harm throughput: triple duplicate ACKs halve the congestion window, while a timeout resets it to one segment, and both force retransmission of lost data. It follows that the true anomaly lies in the

moments just before a congestion event, and it is within that window that the LSTM must learn to find the signal.

To generate the data needed for the LSTM, we utilized the UTCP codebase to generate logs of data transfers. Upon receiving a packet, the UTCP codebase logged both packet details and the system state, most notably the RTT, SRTT, RTTVAR, duplicate ACK count, and congestion window, along with any congestion events. In all, over 252 separate runs of a 1GB transfer from the MacBook Air to the Microsoft VM were collected.

Because the LSTM model requires uniformly spaced inputs, the raw per-packet logs were aggregated into fixed 5ms time bins. A bin is labeled anomalous if it falls within approximately two RTTs before the onset of a congestion event. The central research question is whether an LSTM can be trained to reliably classify bins as anomalous and predict congestion onset before it occurs.

LSTM Development

With the dataset assembled, an LSTM network was designed to intake the binned data and output a per-bin probability of imminent congestion. The model architecture consists of three stacked LSTM layers with unit counts of 512, 512, and 256, preceded by a LayerNormalization step on the input and followed by a 128-unit dense layer with GELU activation, a 20% dropout layer, and a final sigmoid output neuron. In all, the network totals around 4 million tunable parameters. The idea with such a varied and stacked network is that we give the model the best chance to learn the complex relationships that exist in the data.

The eight input features fed into the model at each 5ms bin are: the raw per-bin mean RTT, the Smoothed RTT, RTT variance, queue delay (computed as RTT minus the observed minimum RTT for that connection), the first and second temporal derivatives of RTT, the duplicate ACK count, and the network utilization expressed as the ratio of bytes in flight to the effective window size. All RTT-based features are normalized per-connection by the median baseline RTT, ensuring the model learns relative deviations rather than absolute timing values that would vary across network paths.

For training, the model utilized the Adam optimizer with a learning rate of 3×10^{-4} and a focal loss function in place of standard binary cross-entropy. With positive bins comprising only around 2% of the dataset, the focal loss parameters were tuned to serve two purposes: a focusing

parameter γ of 2.0 downweights the loss contribution from confidently classified negative bins, while a class-weighting parameter α of 0.25 penalizes false positives more heavily than false negatives. This second choice reflects a deliberate design decision, as an overly trigger-happy predictor would cause unnecessary window reductions on healthy connections, degrading throughput without any congestion benefit, so the model was trained from the outset to favor precision over recall.

The dataset was split at the run level into 60% training, 20% validation, and 20% test sets. Splitting by run ensures that the model doesn't learn specific run-contexts between the sets, since consecutive bins within a single connection trace are highly correlated. On the held-out test set, the model was fine-tuned to a threshold of 0.260 and achieved a precision of 0.538, a recall of 0.250, and a final F1 score of 0.341. In other words, the model was about half right on its congestion immediate labels but could only accurately predict about a quarter of congestion labels. Table 1 presents the full per-bin threshold sweep across the test set, illustrating the precision-recall tradeoff as the classification threshold is varied from 0.3 to 0.7.

Threshold	Precision	Recall	F1	TP	FP	FN
0.3	0.691	0.223	0.337	13010	5808	45348
0.4	0.842	0.201	0.325	11746	2197	46612
0.5	0.916	0.177	0.297	10353	947	48005
0.6	0.963	0.143	0.249	8348	322	50010
0.7	0.996	0.095	0.173	5522	23	52836

Table 1: Per-bin threshold sweeps of LSTM results across the test set

However, this measurement must be interpreted carefully. The congestion onset generates a contiguous block of positive-labeled bins spanning the two-RTT window before a congestion event. At the per-window level, where a positive prediction requires only a single alert anywhere within a pre-onset block, the model detected 1,186 of 1,570 congestion windows, achieving a recall of 0.755 and an F1 of 0.566. Of the 1,619 regions in which the model issued an alert, 1,432 were negative regions, or gaps between pre-onset windows where no congestion followed. This

yields a per-window precision of 0.453, meaning roughly half of the regions the model flagged did not precede a congestion event.

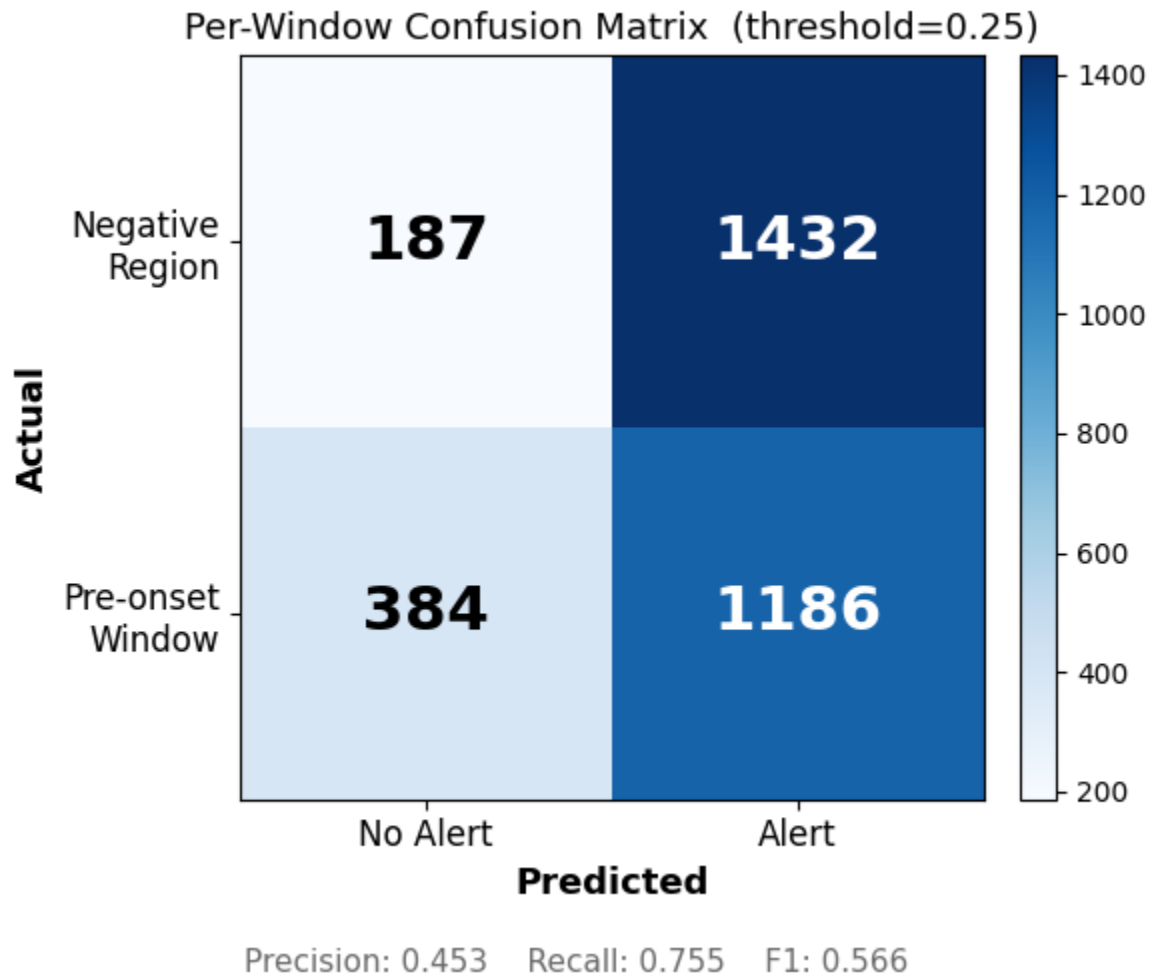


Figure 6: Per-window confusion matrix of the test set at threshold 0.25.

The more striking observation is the inversion of the true negative and false positive counts. Of the 1,619 negative regions in the test set, the model correctly stayed silent in only 187, while raising a fictitious alert in 1,432. This suggests the model is operating in an always alert state at the 0.25 threshold (contrary to how the model was trained and how we expect our model to work).

The gap between per-bin and per-window recall, 0.250 versus 0.755, showcases that within detected pre-onset windows, the model fires sparsely rather than consistently across the full block. Yet, the false positive rate is a significant downturn in performance. 1,432

unnecessary positive labels per test set would translate to unnecessary window reductions on healthy connections, harming throughput more than the congestion events themselves.

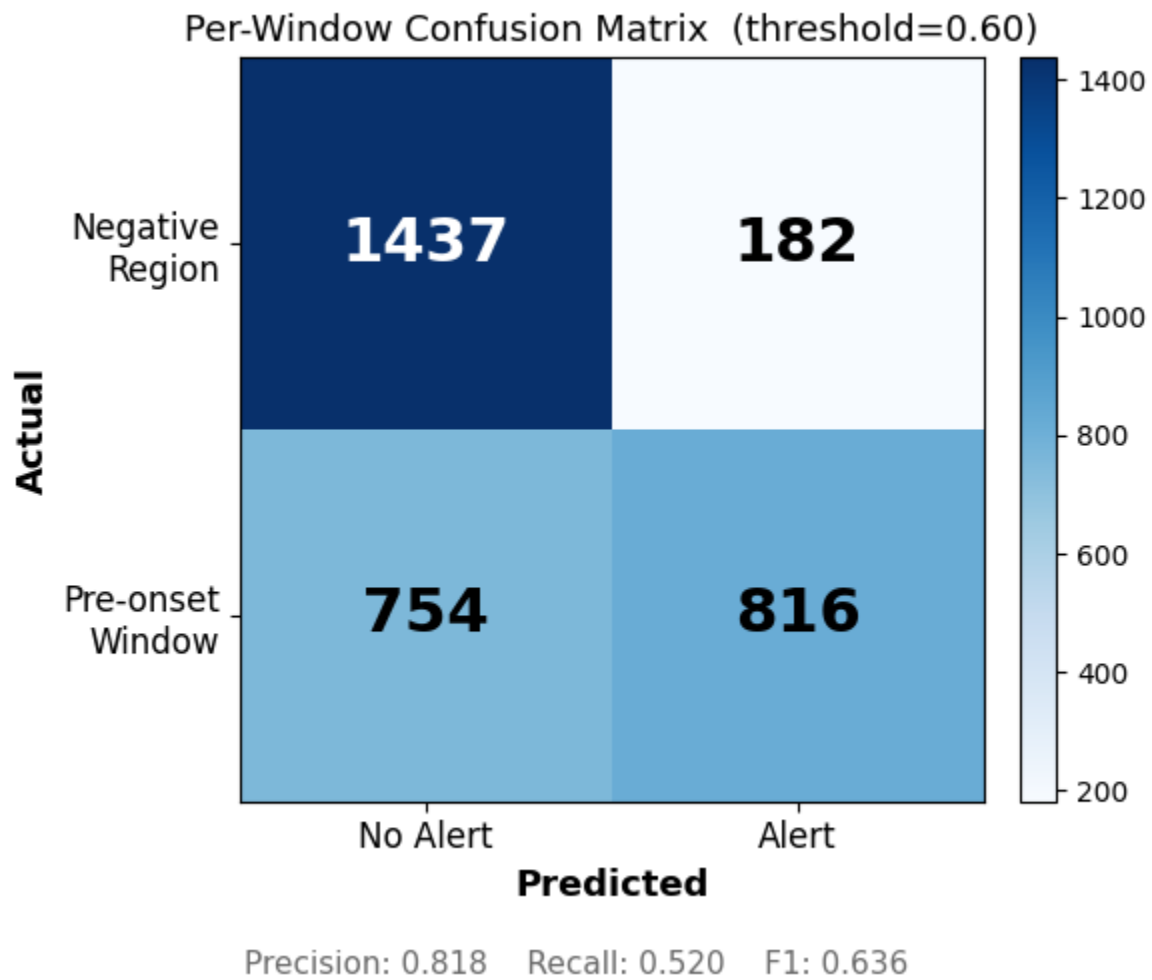


Figure 7: Per-window confusion matrix of the test set at threshold 0.60.

Driving the threshold up to 0.6 yields a precision of 0.818 and a recall of 0.520. While the F1-optimal threshold of 0.25 is the best trade-off of recall and precision, it is unsuitable for deployment. Rather, the 0.6 threshold is the more honest operating point for a CCA integration. A missed prediction, or a false negative, can be recovered by the classical methods for dealing with congestion (i.e., response on triple ack or timeout), without incurring any additional harm. On the other hand, operating at 0.818 precision ensures that we do fire, the model is right four times out of five times, and potentially preventing loss throughput by dynamically adapting our congestion.

Discussion

The model trained here showcases one promising output: the model is picking up on some valid signal. The results of the model cannot simply be noise. A per window recall at threshold 0.6 of 0.520 and a precision of 0.818 significantly exceeds random guessing. This showcases that the eight features carry genuine, learnable information about imminent congestion.

The central limitation of the machine learning, predictive CCA, approach isn't the accuracy, but rather the live inference time. The model was deployed into a live inference pipeline to validate its behavior against real TCP traffic. A lightweight Python server bridged the trained LSTM model and the UTCP stack which communicated over a Unix domain socket. As the stack processes incoming acknowledgments, it forwards the raw connection state to the server, which reconstructs the same 5ms binning and per-connection normalization applied during training and runs inference on each completed bin. The model inference time took inside this server an average of 180ms. This time cost is more than the discrete 5ms bin width time-steps and vastly more time than the two RTT long alert time windows. Hence, this inference time is longer than any predictive CCA could reasonably implement as the model could not propagate it's predictions in time for any congestion control changes to be made.

A simple solution to this would be to decrease the size of the model to reduce the inference time to allow for a wider reaction margin. A smaller model was created with only 120,000 parameters, 15ms bins, and 200 inputs. At a threshold of 0.6, this model achieved per-bin a precision of 0.751 and a recall of 0.132, comparable to the full model's 0.963 precision and 0.143 recall at the same threshold. While a drop in precision and recall may have been lost in the compression, this model still held significant predictive capacity. This small model had an average of 23 ms per inference. While significantly more feasible, as it is less than the 2 RTT time window, the golden standard of this problem would be a inference time shorter or equal to its bin size which this model still fell short of its bin size of 15ms. This would allow the model's predictions to be communicated to the UTCP stack before the next bin was created. This suggests that while large latency gains can be made by decreasing model size, more effort will be needed to polish a model implementation to implement a full, deployable CCA.

This highlights a major issue of machine learning in networking. Network state evolves on the scale of single milliseconds; a single RTT on a local Wi-Fi connection can be as short as

5ms, while even a highly optimized LSTM inference pipeline operating on CPUs operates in the tens to hundreds of milliseconds. There are two different ways to close this gap. Dedicated hardware acceleration, such as a GPU, could bring latency into the sub-millisecond range and is a natural next step to try, but it comes at the cost of a more limited solution. Alternatively, a sufficiently compact model that may achieve the required latency on general hardware, but it remains to be seen if such a model can retain enough capacity to learn the signatures that the large model can only partially capture. Until one of these paths is demonstrated to work consistently at the required timeline, predictive CCAs remain as a research direction as compared to a fully deployable CCA component.

Next Steps

The biggest issue gap in research for these models is the resilience of the model in real-world use case. The dataset was collected on a laptop on a home network communicating with a Microsoft VM. It is very likely that the model learned the characteristics of that network at that specific time point. The single biggest gain a next step could take is to remove the self-collecting data step, and instead, analyze and train on public packet captures of data. This would also address the question of whether there could exist a generalized deep learning model that could be the basis of a predictive CCA, or rather, if predictive CCAs and their models need to be network specific. In addition to this, the model could be a continual learner. In other words, instead of training on static data, the model could be “kept-warm” by being fed new connection traces in production. In this, its hidden state stays apt to the network and context at hand rather than being dataset specific.

A second step could be feature ablation and architecture search. The current model was designed with eight features and a fixed three-layer LSTM architecture. However, it is not currently known which features contributed as helpful information, and which were redundant or noisy. The current hypothesis is that the minimum information needed for a model is only RTT time, as that is the direct signal of congestion. Feature ablation could inform a more compact architecture and allow for a smaller model to still retain the accuracy of the larger one, which directly addresses the latency problem from the model design side.

Conclusion

This project demonstrates that a machine learning model can detect the onset of network congestion before it occurs. The LSTM trained here picks up genuine predictive signals, achieving a per-window precision of 0.818 and recall of 0.520. The barrier to deployment is not accuracy but inference speed, at 180ms per inference, the model is too slow to act on its own predictions. Closing the gap between ML inference timescales and network timescales, whether through hardware acceleration, model compression, or architectural solutions, is the central open problem that stands between predictive congestion control as a research result and predictive congestion control as a deployable system.